WEB APPLICATION SECURITY ASSESSMENT

SecureFolio Penetration Test Report

A hands-on security assessment of a member-portal web application, covering six OWASP Top 10 vulnerability classes — each exploited, remediated, and verified.

1 Critical | 4 High | 1 Medium | 6 / 6 Remediated

Educational lab. SecureFolio is an intentionally vulnerable application built and tested in an isolated, local-only environment owned by the author. No third-party or production system was involved.

AUTHOR	REPORT DATE	TARGET
Adryan	6 November 2026	SecureFolio (localhost)
ASSESSMENT TYPE	**STACK**	**REPOSITORY**
Grey-box web app pentest	Node.js · Express · SQLite	github.com/Adryanrk/web-security-lab

01 Executive Summary

SecureFolio is a member-portal web application (user accounts, private items, and comments). This assessment set out to identify, exploit, and remediate the most common classes of web vulnerability, using the OWASP Top 10 as a framework.

Six distinct vulnerabilities were identified across authentication, access control, injection, data protection, and configuration. One was rated **Critical** (a full authentication bypass via SQL injection) and four **High**, meaning that, as originally built, the application allowed complete account takeover and exposure of every user's private data. All six issues have since been remediated and the fixes verified.

The root cause of most findings was a single recurring theme: **untrusted input being treated as trusted** — whether as SQL code, as HTML, or as an object reference. The remediations apply well-established, low-cost controls: parameterised queries, output encoding, per-request authorization checks, password hashing, generic error handling, and rate limiting.

1	4	1	0
CRITICAL	HIGH	MEDIUM	OPEN



Distribution of the six findings by severity. All findings are remediated; none remain open.

02 Scope & Methodology

Scope. A single target — the SecureFolio application — tested locally at `http://localhost:3000` in an isolated environment. Grey-box: source code was available alongside the running application.

Method. Each vulnerability followed the same loop:

- **Recon** — map pages, forms, URL parameters, and where user input is accepted.
- **Exploit** — confirm each weakness with a concrete, working attack.
- **Assess impact** — determine what an attacker could actually reach or do.
- **Remediate** — fix the issue in code and re-test to confirm the attack fails.
- **Document** — record the vulnerable code, the exploit, and the fix.

Tooling. Browser & DevTools, intercepting proxy (Burp Suite / OWASP ZAP), and purpose-built proof-of-concept scripts (e.g. an automated login brute-forcer). All testing was performed against the author's own local instance.

03 Findings Summary

ID	FINDING	OWASP	SEVERITY	STATUS
F-01	SQL injection in login (authentication bypass)	A03	Critical	Fixed
F-02	Broken access control (IDOR) on item view	A01	High	Fixed
F-03	Stored cross-site scripting (XSS) in comments	A03	High	Fixed
F-04	Passwords stored in plaintext	A02	High	Fixed
F-05	Verbose error pages leak internals	A05	Medium	Fixed
F-06	No rate limiting on login (brute force)	A07	High	Fixed

Severity reflects realistic impact in a production context: Critical = full compromise / all data; High = another user's account or data; Medium = information disclosure enabling further attacks.

04 Detailed Findings

F-01 · SQL Injection in Login

Critical

Fixed

OWASP A03 — Injection · `routes/auth.js` → POST /login

SUMMARY

The login built its SQL query by concatenating user input directly into the command string, so the database could not distinguish code from data. An attacker could rewrite the query and bypass authentication entirely.

EXPLOIT

Submitting the username `' OR 1=1 --` with any password produced:

```
SELECT * FROM users WHERE username = '' OR 1=1 --' AND password = '...'
```

The `--` comments out the password check and `1=1` always matches, returning the first user (admin). Result: login as any account, including admin, with no password.

IMPACT

Complete authentication bypass and administrative access. The same flaw class can typically be extended to dump the entire user table.

REMEDIATION

```
db.prepare('SELECT * FROM users WHERE username = ?').get(username);
```

A parameterised query binds input as data, never as code. The password is then verified separately. **Verified:** the payload is rejected as an ordinary failed login.

F-02 · Broken Access Control (IDOR)

High

Fixed

OWASP A01 — Broken Access Control · `routes/items.js` → GET /items/:id

SUMMARY

The item page fetched an object by the id in the URL and displayed it without checking ownership. Any logged-in user could read other users' private items by changing the number in the address.

EXPLOIT

Logged in as a normal user (alice), changing `/items/1` to `/items/2` and `/items/3` exposed Bob's and the administrator's private items.

IMPACT

Horizontal privilege escalation and bulk data exposure — an attacker can enumerate `/items/1..N` to extract all records.

REMEDIATION

```
if (item.owner_id !== req.session.userId && req.session.role !== 'admin')
  return res.status(403).render('403');
```

Authorization is now enforced per object: authentication proves *who* you are; this check proves you are entitled to *this* resource. **Verified:** cross-user access now returns 403.

F-03 · Stored Cross-Site Scripting

OWASP A03 — Injection (XSS) · `views/item.ejs` → comment rendering

High

Fixed

SUMMARY

Comments were rendered as raw, unescaped HTML. A comment containing a script was stored and then executed in the browser of every user who viewed the item — persistent (stored) XSS.

EXPLOIT

A comment of `<script>…</script>` submitted by a normal user executed in the administrator's browser when they viewed the item, demonstrating low-to-high-privilege attack delivery.

IMPACT

Code execution in the victim's session: acting as the victim, defacing content, or redirecting them. With an admin victim, a path to account takeover. (The session cookie is `httpOnly`, which limits cookie theft but does not make the flaw safe.)

REMEDIATION

Render user content with EJS's escaped output tag (`<%= %>` rather than `<%- %>`) so stored HTML is shown as text, never executed. Defence in depth: a Content-Security-Policy header and HTML sanitisation for rich content. **Verified:** payloads now display as inert text.

F-04 · Passwords Stored in Plaintext

OWASP A02 — Cryptographic Failures · `db/db.js`, `routes/auth.js`

High

Fixed

SUMMARY

User passwords were stored as plaintext. Any database exposure — via injection, a stolen backup, or a misconfiguration — would hand an attacker every password, immediately usable, with password reuse compromising users' other accounts.

EVIDENCE

```
id  username  password      role
1   admin     Admin!2026    admin
2   alice     alice123      user
```

IMPACT

Total credential disclosure on any leak, with no cracking required; administrator credentials exposed.

REMEDIATION

```
bcrypt.hashSync(password, 10);          // store the hash
bcrypt.compareSync(input, user.password) // verify at login
```

Passwords are now bcrypt hashes — one-way, uniquely salted, and deliberately slow to resist brute force. The plaintext is never stored. **Verified:** the database now holds only hashes.

F-05 · Verbose Error Pages Leak Internals

Medium

Fixed

OWASP A05 — Security Misconfiguration · `server.js` → error handler

SUMMARY

Unhandled errors returned the full stack trace to the client, leaking absolute file paths, the OS account, the framework, and the code structure — a map an attacker uses to fingerprint and target the application. No authentication was required to trigger it.

IMPACT

Information disclosure that accelerates further attacks; an attacker deliberately provokes errors to harvest internal detail.

REMEDIATION

```
console.error(err);              // details to server logs only
res.status(500).render('500');   // generic page to the user
```

Diagnostics are now logged server-side only; users see a generic page. Debug/test endpoints were removed. **Verified:** errors return the generic 500 page. Recommended hardening: `NODE_ENV=production` and `app.disable('x-powered-by')`.

F-06 · No Rate Limiting on Login

High

Fixed

OWASP A07 — Identification & Authentication Failures · `routes/auth.js` → POST /login

SUMMARY

The login accepted unlimited attempts, allowing automated brute force and password spraying. A proof-of-concept script cracked a test account in ten guesses with no resistance.

EXPLOIT

```
[10] alice:alice123 -> SUCCESS
Cracked in 10 attempts — no blocking.
```

IMPACT

Account takeover of any account with a guessable password; the login's other protections do not help if guessing is unlimited.

REMEDIATION

```
rateLimit({ windowMs: 15*60*1000, limit: 5 }) // per IP
```

The login is now limited to five attempts per IP per window, returning `429 Too Many Requests` thereafter. **Verified:** the same script is now blocked after five attempts. Recommended defence in depth: account lockout/backoff, MFA, and breached-password screening.

05 Conclusion

As originally built, SecureFolio was exploitable to the point of full compromise: an attacker could bypass login, read any user's private data, run code in other users' browsers, and recover every password from a single database leak.

All six findings have been remediated using standard, well-understood controls, and each fix was re-tested against the original exploit to confirm it holds. No findings remain open.

The exercise reinforces a small set of durable principles that prevent most of these issues: never trust input (bind it as data, escape it on output), authorize every request per object rather than only authenticating the user, store secrets as salted one-way hashes, keep internal detail out of responses, and limit how often sensitive actions can be attempted.

SecureFolio is an educational project. It was built intentionally vulnerable and tested only against the author's own local instance. The techniques described must never be used against systems without explicit authorisation. Full source, per-finding write-ups, and proof-of-concept code: github.com/Adryanrk/web-security-lab